

Evaluación Parcial N°2

Encargo con Presentación

Informe técnico de proyecto de datos

Pipeline DataOps para Detección de Fraude con Tarjetas de Crédito

Campo	Detalle
Asignatura	Gestión de Datos para IA
Sigla	ITY1101
Caso seleccionado	03 Credit Card Fraud
Dataset base	02_fraudTest.csv
Arquitectura implementada	Lakehouse Bronze/Silver/Gold
Backend / API	FastAPI + Python
Dashboard	https://fraud-pipeline.onrender.com/
Repositorio GitHub	risaavedraf/Gestion_de_datos_para_ia
Sección	01D
Docente	Franco Jorquera
Integrantes	Roberto Armijo, Miguel Calderón, Ricardo Saavedra
Fecha	20/05/2026

Documento revisado para concordar con la implementación real del repositorio, la página de demo y la rúbrica del informe.

Índice

- 1. Resumen ejecutivo
- 2. Diagnóstico del caso y valor para la organización
- 3. Justificación de la metodología PMBOK aplicada
- 4. Planificación del proyecto
- 5. Diseño e implementación del pipeline DataOps
 - 5.1 Arquitectura Lakehouse Bronze/Silver/Gold
 - 5.2 Ingesta de datos - Capa Bronze
 - 5.3 Limpieza y transformación - Capa Silver
 - 5.4 Validación estructural y semántica - Capa Gold
 - 5.5 Carga de datos a PostgreSQL
 - 5.6 Capa analítica y modelo de detección de fraude
- 6. Plan de seguridad del entorno DataOps
- 7. Evidencias de implementación, CI/CD y demo funcional
- 8. Estrategia de KPIs de monitoreo
- 9. Cumplimiento de rúbrica del informe
- 10. Conclusiones y próximos pasos
- 11. Referencias

1. Resumen ejecutivo

El presente informe documenta el diseño e implementación de un pipeline DataOps aplicado al caso Credit Card Fraud, orientado al tratamiento de transacciones con tarjeta de crédito y a la preparación de datos confiables para análisis, monitoreo y una demo funcional de detección de fraude.

El dataset base corresponde al archivo 02_fraudTest.csv, con 555.719 registros y 23 columnas originales. La variable objetivo es is_fraud, donde 1 representa una transacción fraudulenta y 0 una transacción legítima. La revisión inicial muestra un escenario altamente desbalanceado: 553.574 transacciones legítimas y 2.145 fraudulentas, equivalente a una tasa de fraude aproximada de 0,386%.

El foco del proyecto no se limita al entrenamiento de un modelo, sino a asegurar que los datos pasen por un flujo ordenado, trazable, reproducible y seguro. Para ello se implementa una arquitectura Lakehouse con capas Bronze, Silver y Gold, seguida de carga persistente en PostgreSQL, exposición mediante una API FastAPI y visualización en un dashboard web desplegado en Render.

Elemento	Detalle aplicado al proyecto
Proyecto	Pipeline DataOps para detección de fraude con tarjetas de crédito
Fuente	CSV 02_fraudTest.csv del caso 03_Credit_Card_Fraud
Arquitectura	Lakehouse: Data/bronze, Data/silver, Data/gold y Data/rejected
Backend	FastAPI con endpoints /api/* para pipeline, dataset, KPIs, validación y modelo
Base de datos	PostgreSQL con tablas relacionales e integridad referencial
Demo	Dashboard web en Render y ejecución local con Docker Compose

2. Diagnóstico del caso y valor para la organización

El caso Credit Card Fraud representa una problemática común en instituciones financieras, fintech, comercios digitales y entidades que procesan pagos. Estas organizaciones necesitan identificar operaciones sospechosas, pero para hacerlo de manera confiable primero requieren datos consistentes, completos, protegidos y correctamente estructurados.

El principal desafío del caso es el desbalance de clases. Si solo se observa la exactitud general, un sistema podría parecer correcto aun cuando no detecte fraudes. Por eso, la solución debe monitorear la distribución de fraude y considerar métricas adecuadas en la etapa analítica, como recall, precision, F1-score, matriz de confusión y ROC-AUC.

Además, el dataset contiene campos asociados a comportamiento transaccional, ubicación, edad, comercio, monto y número de tarjeta. Aunque el contexto sea académico, el pipeline debe tratar estos datos como sensibles: no se deben exponer números completos de tarjeta en logs, capturas o demo, y las credenciales no deben subirse al repositorio.

Problema detectado	Impacto posible	Respuesta implementada con DataOps
Datos financieros y personales	Riesgo de exposición de información sensible	Enmascaramiento de cc_num, uso de .env, .gitignore y logs seguros
Dataset altamente desbalanceado	Fraudes no detectados si se mide solo accuracy	KPIs de distribución y métricas de modelo orientadas a fraude
Proceso manual no trazable	Dificultad para auditar errores y repetir resultados	Logs, metadata JSON, Git, README y capas Bronze/Silver/Gold
Carga masiva de datos	Duplicaciones o fallas de integridad	Deduplicación, claves primarias y ON CONFLICT en PostgreSQL
Demo en ambiente externo	Dependencia de disponibilidad del hosting	Render como demo web y respaldo local con docker-compose up -d

3. Justificación de la metodología PMBOK aplicada

Para este proyecto se utiliza una metodología mixta basada en PMBOK. Esta decisión se justifica porque el trabajo combina actividades predictivas, como definición de alcance, entregables, fechas, responsabilidades y evidencias, con actividades adaptativas, como ajustes de reglas de limpieza, validación, manejo de datos rechazados y decisiones técnicas según los hallazgos del dataset.

La parte predictiva permite ordenar el proyecto dentro del tiempo disponible de la evaluación. Se definen productos concretos: scripts del pipeline, estructura Lakehouse, base de datos, API, dashboard, CI/CD, Dockerfile, README, informe y presentación. La parte adaptativa permite corregir reglas cuando aparecen anomalías, por ejemplo valores fuera de rango, errores de tipo, duplicados por trans_num o diferencias entre fechas.

Enfoque	Aplicación en el proyecto
Predictivo	Define alcance, responsables, fechas, entregables, repositorio, demo y evidencias requeridas por la rúbrica.

Adaptativo	Permite ajustar reglas de limpieza, validación, carga y monitoreo según los resultados de ejecución.
Mixto	Mantiene control formal del proyecto sin perder flexibilidad técnica frente a datos reales o semirrealistas.

4. Planificación del proyecto

La planificación se organiza mediante una WBS con entregables verificables. Para el seguimiento se recomienda GitHub Projects o Trello, complementado con commits descriptivos, issues y control de avance en el repositorio. GitHub Actions refuerza la revisión técnica al ejecutar pruebas e importaciones antes de integrar cambios.

WBS	Actividad	Entregable/evidencia	Responsable sugerido
1.1	Revisión del caso y dataset	Ficha técnica del dataset y diagnóstico inicial	Equipo
1.2	Estructura del repositorio	Carpetas backend, frontend, Data, Docs y workflows	Miguel Calderón
2.1	Ingesta - Bronze	backend/src/ingestion.py, fraud_bronze.parquet y metadata	Ricardo Saavedra
2.2	Limpieza - Silver	backend/src/cleaning.py, fraud_silver.parquet y metadata	Ricardo Saavedra
2.3	Validación - Gold	backend/src/validation.py, validation_report.json y rechazados	Roberto Armijo
2.4	Carga a PostgreSQL	backend/src/loader.py y tablas relacionales	Equipo
3.1	API y dashboard	FastAPI, endpoints /api/* y frontend SPA	Equipo
3.2	Seguridad y configuración	.env.example, .gitignore, enmascaramiento y logs seguros	Equipo
3.3	CI/CD y despliegue	Dockerfile, docker-compose, GitHub Actions y Render	Equipo
4.1	Informe y presentación	PDF, Word editable, presentación y demo funcional	Equipo

Semana	Actividades principales	Evidencia esperada
Semana 1	Diagnóstico, estructura del repositorio, ingesta Bronze y exploración inicial.	Repositorio, dataset en Data/bronze, log y metadata de ingesta.
Semana 2	Limpieza Silver, validación Gold, reportes y reglas de calidad.	Datasets silver/gold, validation_report.json, rechazados y commits.
Semana 3	Carga en PostgreSQL, API, dashboard, CI/CD, Docker, Render y documentación.	Base poblada, dashboard, endpoints, workflow CI, README e informe final.

5. Diseño e implementación del pipeline DataOps

El pipeline implementado sigue una arquitectura secuencial y modular: CSV -> Bronze -> Silver -> Gold -> PostgreSQL -> Dashboard, con una rama analítica para entrenamiento y predicción de fraude. Cada etapa tiene una entrada definida, una salida verificable y evidencia mediante metadata, logs o reportes.

Etapla académica	Implementación real	Entrada	Salida
Ingesta	Bronze: backend/src/ingestion.py	02_fraudTest.csv	Data/bronze/fraud_bronze.parquet + ingestion_metadata.json

Limpieza y transformación	Silver: backend/src/cleaning.py	Bronze Parquet	Data/silver/fraud_silver.parquet + cleaning_metadata.json
Validación estructural y semántica	Gold: backend/src/validation.py	Silver Parquet	Data/gold/fraud_gold.parquet + validation_report.json
Carga	PostgreSQL: backend/src/loader.py	Gold Parquet	customers, merchants, transactions, logs y tablas de modelo
Uso posterior	API + Dashboard + ML	PostgreSQL/Gold	KPIs, visualizaciones, entrenamiento y predicción

5.1 Arquitectura Lakehouse Bronze/Silver/Gold

La arquitectura Lakehouse permite separar el ciclo de vida del dato por niveles de madurez. Bronze conserva el dato crudo e inmutable, Silver contiene datos limpios y transformados, Gold almacena el dataset validado y curado, y PostgreSQL permite persistencia relacional para consultas y demo. Esta estructura evita mezclar datos originales con datos modificados y facilita explicar qué ocurrió en cada etapa.

Capa	Propósito	Carpeta/archivo real
Bronze	Conservar el dataset crudo, calcular checksum y registrar metadata.	Data/bronze/fraud_bronze.parquet, fraud_bronze.csv, ingestion_metadata.json
Silver	Limpiar, convertir tipos, normalizar texto y crear variables derivadas.	Data/silver/fraud_silver.parquet, fraud_silver.csv, cleaning_metadata.json
Gold	Aplicar validaciones estructurales y semánticas, separando registros rechazados.	Data/gold/fraud_gold.parquet, fraud_gold.csv, validation_report.json
Rejected	Guardar registros que fallan reglas de validación y sus motivos.	Data/rejected/fraud_rejected.parquet

5.2 Ingesta de datos - Capa Bronze

La ingesta es el punto de entrada del pipeline. En la implementación real, el script backend/src/ingestion.py lee el CSV base, verifica su existencia, calcula un checksum SHA256, registra metadata y guarda una versión en Parquet y CSV dentro de Data/bronze. El dato original se mantiene como referencia para asegurar trazabilidad y repetibilidad.

Decisión técnica: usar Parquet como formato principal reduce tiempos de lectura y conserva tipos de forma más eficiente, mientras que CSV se mantiene como respaldo de fácil inspección durante la demo o revisión académica.

5.3 Limpieza y transformación - Capa Silver

La limpieza y transformación se implementa en backend/src/cleaning.py. El proceso elimina columnas técnicas como Unnamed: 0, convierte cc_num a texto, transforma fechas, normaliza variables categóricas como category, gender, state, merchant y city, y crea variables derivadas útiles para análisis de fraude.

Entre las variables derivadas se encuentran trans_hour, trans_day_of_week, trans_month, age_at_transaction, distance_km y cc_num_masked. Además, se eliminan columnas sensibles como first, last,

street y dob para reducir exposición de datos personales. Los montos extremos no se eliminan automáticamente, ya que pueden ser relevantes para detectar transacciones sospechosas.

Transformación implementada	Justificación
Eliminar Unnamed: 0	Evita cargar un índice técnico como variable de negocio.
cc_num como texto y cc_num_masked	Evita pérdida de formato y reduce exposición de tarjeta.
Normalizar strings	Disminuye errores por diferencias de mayúsculas, espacios o formatos.
Crear variables temporales	Permite analizar patrones por hora, día y mes.
Calcular distance_km	Una distancia atípica cliente-comercio puede aportar señal de riesgo.
Eliminar first, last, street y dob	Aplica minimización de datos personales en la capa procesada.

5.4 Validación estructural y semántica - Capa Gold

La validación se implementa en backend/src/validation.py. La validación estructural comprueba columnas obligatorias, tipos de datos, unicidad de trans_num, dominio de is_fraud y rangos numéricos/geográficos. La validación semántica revisa reglas del negocio, como que el monto sea mayor a cero, que las coordenadas sean posibles y que las categorías pertenezcan al catálogo esperado.

Una decisión importante es registrar ciertas inconsistencias temporales como advertencias cuando no justifican rechazar el registro completo. Esto evita eliminar datos útiles de forma innecesaria y demuestra criterio técnico en el manejo de anomalías.

Tipo	Regla aplicada al caso	Acción si falla
Estructural	Columnas obligatorias presentes	Reporte de error y bloqueo del flujo si falta estructura crítica
Estructural	trans_num único	Rechazo o deduplicación para evitar doble carga
Estructural	amt numérico y mayor a 0	Registro rechazado o enviado a revisión
Semántica	is_fraud solo puede ser 0 o 1	Rechazo por dominio inválido
Semántica	lat/long y merch_lat/merch_long dentro de rangos válidos	Rechazo por coordenadas imposibles
Semántica	category dentro del catálogo del dataset	Registro de categoría desconocida

5.5 Carga de datos a PostgreSQL

La carga se implementa en backend/src/loader.py desde la capa Gold hacia PostgreSQL mediante SQLAlchemy. El proceso crea o valida tablas, deduplica entidades y utiliza reglas de conflicto para evitar inserciones repetidas. Esta descripción reemplaza la idea inicial de una tabla staging, ya que la implementación real resuelve la carga controlada con deduplicación, claves primarias y ON CONFLICT.

Tabla	Propósito	Clave / control
customers	Datos anonimizados o minimizados del titular.	customer_id generado desde cc_num hasheado
merchants	Comercios y categoría asociada.	merchant_id y UNIQUE(merchant_name, category)

transactions	Tabla principal de transacciones, monto, tiempo, ubicación y etiqueta fraude.	trans_num como clave primaria
pipeline_logs	Historial de ejecución por etapa.	run_id, stage, status y duración
rejected_records	Registros rechazados con motivo.	reject_id y run_id
model_predictions	Predicciones generadas por el modelo.	prediction_id y trans_num
model_training_runs	Historial de entrenamientos y métricas.	training_run_id

5.6 Capa analítica y modelo de detección de fraude

Aunque la rúbrica del informe se centra en DataOps, el proyecto implementa una capa analítica posterior. El backend incluye módulos de feature engineering, entrenamiento, evaluación y predicción: `features.py`, `model_train.py`, `model_evaluate.py` y `model_predict.py`. Esta capa permite demostrar el valor del pipeline: los datos limpios y validados alimentan una demo de detección de fraude y métricas de modelo.

Debido al desbalance de clases, la evaluación del modelo no debe depender solamente de accuracy. Para explicar el desempeño se recomiendan recall, precision, F1-score, matriz de confusión y ROC-AUC, priorizando la capacidad de detectar fraudes sin ocultar falsos positivos o falsos negativos.

6. Plan de seguridad del entorno DataOps

El proyecto se diseña considerando que los datos financieros y personales requieren controles desde el inicio. Aunque el dataset sea académico, contiene campos que representan tarjeta, ubicación, identidad, edad, ocupación, monto y comportamiento transaccional. Por ello, se aplican medidas de minimización, enmascaramiento, control de acceso y gestión segura de credenciales.

En Chile se considera como marco de referencia la Ley N° 19.628 sobre protección de datos personales y la Ley N° 21.719, que moderniza el tratamiento de datos personales y crea la Agencia de Protección de Datos Personales. Para el contexto de tarjetas se considera PCI DSS v4.0.1 como referencia técnica internacional para proteger datos de pago.

Medida	Aplicación concreta en el proyecto
Control de acceso	Solo integrantes autorizados modifican repositorio, base de datos y variables de entorno.
Variables de entorno	Credenciales fuera del código mediante <code>.env</code> ; en el repo solo se publica <code>.env.example</code> .
Enmascaramiento	cc_num no se muestra completo; se usa cc_num_masked o hash para identificación técnica.
Minimización	Se eliminan first, last, street y dob en la capa procesada cuando no son necesarios.
Logs seguros	Los logs registran cantidades, errores y estados sin exponer tarjetas ni direcciones completas.
<code>.gitignore</code>	Evita publicar credenciales, archivos temporales o datos sensibles no necesarios.
Cifrado	Se recomienda conexión segura en tránsito y protección de backups o exports en reposo.
Demo controlada	La demo web puede usar muestra reducida o sintética para no exponer el dataset completo.

7. Evidencias de implementación, CI/CD y demo funcional

La implementación cuenta con evidencias técnicas alineadas con la rúbrica: repositorio GitHub, scripts del pipeline, README, Dockerfile, docker-compose.yml, workflow de GitHub Actions, backend FastAPI, frontend dashboard, reportes JSON, logs y despliegue en Render.

Evidencia	Qué demuestra	Ubicación real
Código de ingesta	Lectura del CSV, checksum, Parquet y metadata.	backend/src/ingestion.py
Código de limpieza	Conversión de tipos, normalización y variables derivadas.	backend/src/cleaning.py
Código de validación	Reglas estructurales, semánticas, Gold y rechazados.	backend/src/validation.py
Código de carga	PostgreSQL, creación de tablas, deduplicación y carga.	backend/src/loader.py
Backend API	Endpoints para pipeline, dataset, KPIs, validación y modelo.	backend/app.py
Frontend SPA	Dashboard con Overview, Dataset, Architecture, Pipeline, PMBOK, Security, CI/CD, Model y Demo.	frontend/
CI/CD	Instala dependencias, ejecuta pytest y verifica imports.	.github/workflows/ci.yml
Docker	Ejecución reproducible del proyecto.	Dockerfile y docker-compose.yml
Demo web	Visualización y prueba funcional del proyecto.	https://fraud-pipeline.onrender.com/

Para la presentación se recomienda ejecutar una muestra del pipeline para evitar demoras. La demo debe mostrar la estructura del repositorio, ejecución de etapas, visualización de KPIs, consulta de distribución de fraude y funcionamiento del endpoint o formulario de predicción. Si Render está suspendido o responde con error temporal, se debe tener preparado el respaldo local con docker-compose up -d y acceso a <http://localhost:8000>.

8. Estrategia de KPIs de monitoreo

Los KPIs permiten verificar si el pipeline mantiene calidad, eficiencia y confiabilidad. En la implementación, estos indicadores pueden exponerse mediante el endpoint `/api/kpis` y visualizarse desde el dashboard web, complementándose con reportes de validación y logs de ejecución.

KPI	Cómo se mide	Meta sugerida	Alerta
Compleitud	Registros sin nulos críticos / total procesado	>= 99%	Baja de 98%
Duplicados	Duplicados por trans_num / total	0%	Aparece cualquier duplicado
Tasa de rechazo	Registros rechazados / total procesado	<= 5%	Supera 5%
Tiempo de ejecución	Fin - inicio del pipeline	<= 15 min en muestra académica	Aumenta más de 30%
Éxito de carga	Insertados / validados	>= 99%	Fallas de inserción
Distribución de fraude	% is_fraud = 1	Referencia aprox. 0,386%	Cambio abrupto
Errores por tipo	Conteo por causa de rechazo	Tendencia estable o baja	Error repetido muchas veces
Disponibilidad	Existencia de salidas Bronze/Silver/Gold/logs	100%	Falta archivo o endpoint

9. Cumplimiento de rúbrica del informe

La siguiente tabla resume cómo el informe cubre los indicadores del encargo. Se agrega para que la documentación no pierda el foco evaluativo y permita defender el trabajo frente a preguntas técnicas.

Indicador de rúbrica del informe	Dónde se evidencia en este documento
Identifica problema clave y justifica DataOps con PMBOK	Secciones 2 y 3: diagnóstico del fraude, desbalance, valor organizacional y metodología mixta.
Documenta el pipeline DataOps etapa por etapa	Sección 5: Bronze, Silver, Gold, carga PostgreSQL y capa analítica.
Lista políticas básicas de seguridad DataOps	Sección 6: control de acceso, .env, enmascaramiento, minimización, cifrado y leyes.
Documenta evidencias de implementación	Sección 7: repo, scripts reales, API, dashboard, Docker, CI/CD, Render y respaldo local.
Propone KPIs críticos de monitoreo	Sección 8: completitud, duplicados, rechazo, ejecución, carga, fraude y disponibilidad.

10. Conclusiones y próximos pasos

El proyecto demuestra una aplicación concreta de DataOps en un caso financiero. En lugar de tratar el fraude únicamente como un problema de modelamiento, se aborda primero la base del proceso: ingesta, limpieza, validación, carga, monitoreo, seguridad y documentación. Esta mirada es clave porque la calidad de cualquier modelo depende directamente de la calidad y trazabilidad de los datos que recibe.

La implementación real del repositorio fortalece el informe porque incorpora arquitectura Lakehouse, PostgreSQL, FastAPI, dashboard web, Docker, GitHub Actions, Render y módulos de machine learning. Estos elementos permiten defender el proyecto no solo como una planificación, sino como una solución ejecutable y demostrable.

Como próximos pasos se propone reforzar pruebas unitarias de reglas de validación, mejorar la visualización de alertas, documentar capturas de la demo, mantener un dataset de muestra seguro para presentación y profundizar la evaluación del modelo con métricas adecuadas para clases desbalanceadas.

11. Referencias

Duoc UC. (2025). Evaluación Parcial N°2 - Encargo con Presentación, ITY1101 Gestión de Datos para IA.

Duoc UC. (2025). Pipeline de Datos - Ingesta de datos automatizada. Material de clases ITY1101.

Duoc UC. (2025). Pipeline de Datos - Limpieza y transformación del dataset. Material de clases ITY1101.

Duoc UC. (2025). Pipeline de Datos - Validación estructural y semántica de datos. Material de clases ITY1101.

Duoc UC. (2025). Pipeline de Datos - Carga de los datos a Base de Datos. Material de clases ITY1101.

Repositorio del proyecto. Gestión de datos para IA - Pipeline DataOps Credit Card Fraud Detection.
https://github.com/risaavedraf/Gestion_de_datos_para_ia

Demo web del proyecto. Fraud Pipeline. <https://fraud-pipeline.onrender.com/>

Biblioteca del Congreso Nacional de Chile. Ley N° 19.628 sobre protección de la vida privada.

Biblioteca del Congreso Nacional de Chile. Ley N° 21.719 sobre protección y tratamiento de datos personales.

PCI Security Standards Council. PCI Data Security Standard (PCI DSS) v4.0.1.